

m1

Hydrology Systems in 3D Engines: Sea Level Logic, Wave Simulation, and Dynamic Coastline Shading

The Core Concept: The Infinite Horizon

A hydrology system in a 3D engine is typically composed of two interacting layers: a static reference plane representing sea level and a dynamic displacement system that simulates waves. Together, these create the illusion of an infinite, living ocean surface interacting with terrain geometry.

The sea level acts as a global height threshold applied to all terrain vertices. Any vertex with a Y-value below this threshold is classified as underwater, while vertices above it remain part of the visible landmass. This simple comparison forms the foundation of water-land segmentation in procedural environments.

<https://www.udemy.com/course/web-based-3d-terrain-industrial-printing-engine-mastery/>

Mathematical Baseline for Water Classification

The classification of terrain vertices relative to sea level can be expressed as a conditional function that determines rendering behavior based on height.

```
f(y) = {  
    Underwater Shading    if y < SeaLevel  
    Coastal/Land Shading  if y ≥ SeaLevel  
}
```

This logic is not only visual but also structural, as it can influence physics, post-processing effects, and even sound simulation layers.

To avoid a static, glass-like water surface, wave motion is generated using sinusoidal functions applied over vertex positions. Each vertex receives a displacement based on both its spatial coordinates and elapsed time, producing continuous motion across the surface.

The wave function typically combines sine and cosine components to create multi-directional interference patterns, simulating natural ocean complexity.

$$y = \sin(x \times \text{frequency} + \text{time}) \times \cos(z \times \text{frequency} + \text{time}) \times \text{amplitude}$$

By varying frequency and amplitude, the system can simulate anything from calm lakes to turbulent seas. The inclusion of vertex position in the phase calculation ensures that waves are spatially coherent rather than uniformly oscillating.

Trainee Assignments: Technical Tasks

Task 1: The Flood Simulation

The system includes a controllable `seaLevel` parameter that defines the global water height.

The task is to increase this value until only the highest peaks of the terrain remain above water. This simulates a flooding scenario where terrain is progressively submerged.

While doing this, observe how vertex coloring behaves at the boundary between land and water. The key question is whether shoreline colors dynamically follow the water surface or remain fixed to static height thresholds. This determines whether the system is truly reactive or precomputed.

Task 2: Tidal Frequency Modification

Inside the animation loop, the wave update logic includes a frequency multiplier that controls spatial wave density.

The task is to change this value from 0.05 to 0.5 and observe the result.

Increasing frequency compresses wave spacing, creating a choppy, small-scale water surface. Lower frequency produces large, slow undulations that feel more like deep ocean behavior. This demonstrates how frequency directly controls perceived environmental scale in procedural rendering systems.

Assessment Questions: Hydrology Logic

Question 1: Transparency vs Transmission

In physically based rendering systems such as MeshPhysicalMaterial, opacity and transmission represent fundamentally different optical behaviors.

Opacity simply reduces visibility by fading pixels, while transmission simulates the actual passage of light through a medium. Transmission accounts for refraction, scattering, and internal light transport, making it essential for realistic water rendering where light bends and diffuses through liquid volumes rather than simply disappearing.

Question 2: The STL Export Conflict

When exporting terrain for 3D printing, the sea layer is typically excluded because it violates manifold geometry requirements.

3D printers require watertight meshes without internal planes that do not contribute to solid volume. A floating sea surface introduces non-manifold geometry, which breaks slicing algorithms. Therefore, only the terrain mesh is exported, ensuring structural consistency and printable geometry.

Question 3: Vertex Interaction and Sediment Simulation

The seaLevel value can be used not only for visualization but also for terrain modification. When vertices fall below sea level, they can be gradually adjusted upward or flattened to simulate sediment deposition.

This creates a feedback loop where underwater terrain slowly levels out over time, mimicking natural sediment accumulation processes found in real-world hydrology systems.

Advanced Challenge: Dynamic Coastlines

In most basic systems, coastal shading is based on fixed height thresholds. However, this approach fails when sea level becomes dynamic.

vertex in real time based on its distance from the evolving water surface.

Conceptual Approach

A dynamic coastal zone can be defined as:

- Vertices near seaLevel $\pm$ a small offset range
- Continuously recalculated as seaLevel changes

This produces a moving shoreline that adapts in real time.

Implementation Strategy

Rather than storing fixed height bands, the shader or CPU logic should compute:

- Absolute difference between vertex height and seaLevel
- Apply gradient-based blending for coastal shading

This ensures that beaches "follow" the water level dynamically, enabling fully responsive environmental transitions without hardcoded terrain values.

Final Insight

Hydrology systems in 3D engines are not just visual effects but dynamic classification systems. By combining height thresholds, trigonometric wave motion, and relative shading logic, it becomes possible to simulate fully responsive ocean-terrain interactions where coastlines, waves, and sediment behavior evolve continuously in real time.

m2

m3